POSTECH

Pós-Graduação em Arquitetura e Desenvolvimento Java

TECH CHALLENGE — FASE 1

Restaurant Management System

Relatório Técnico

Backend em Spring Boot 3.3 com arquitetura hexagonal, autenticação JWT e PostgreSQL

Equipe

Gabriel Andrade Almeida - RM371308

Maio de 2026

Sumário

1. Introdução....4
1.1 Descrição do problema....4
1.2 Objetivo do projeto....4
1.3 Metodologia de desenvolvimento....5
2. Arquitetura do Sistema....6
2.1 Estilo arquitetural....6
2.2 Camadas e responsabilidades....6
2.3 Diagrama da arquitetura....6
2.4 Fluxo de uma requisição....7
2.5 Wiring de beans: domain agnóstico ao Spring....8
2.6 Estratégia plugável de autenticação....9
2.7 Tratamento de erros: ProblemDetail (RFC 7807)....9
2.8 Estrutura de pacotes....10
3. Modelagem de Dados....11
3.1 Modelo de entidade....11
3.2 Diagrama Entidade-Relacionamento....11
3.3 Migrações Flyway....12
3.4 Decisões importantes....12
4. Descrição dos Endpoints da API....13
4.1 Tabela de endpoints....13
4.2 Exemplos de requisição e resposta....13
4.2.1 POST /api/v1/users — cadastro de usuário....13
4.2.2 POST /api/v1/auth/login — autenticação....14
4.2.3 PATCH /api/v1/users/{id}/password — alteração de senha....15
4.2.4 GET /api/v1/users?name={termo} — busca por nome....16
4.2.5 Demais endpoints....16
4.3 Códigos HTTP utilizados....16
5. Configuração do Projeto....18
5.1 Stack e versões....18
5.2 Profiles Spring....18
5.3 Configuração do Docker Compose....18
5.4 Dockerfile multi-stage....19
5.5 Variáveis de ambiente....20
5.6 Instruções para execução....20
5.6.1 Execução local com profile dev (H2)....20
5.6.2 Execução com Docker Compose (PostgreSQL real)....21
5.6.3 Resetar dados entre execuções....21
6. Qualidade do Código e Testes....22
6.1 Boas práticas adotadas....22
6.2 Estratégia de testes....22

6.3 ArchUnit: 8 regras como guardião da arquitetura....23
6.4 Padrões específicos descobertos durante o desenvolvimento....23
7. Documentação Swagger / OpenAPI.... 25
7.1 Configuração....25
7.2 Anotações em controllers e DTOs....25
7.3 Acesso à interface....25
7.4 Fluxo de uso pelo Swagger UI....25
7.5 Validação automatizada.... 26
8. Coleção Postman para Testes....27
8.1 Estrutura da coleção....27
8.2 Variáveis de ambiente.... 27
8.3 Importação.... 28
8.4 Execução com Newman (linha de comando)....28
8.5 Cobertura dos critérios de aceite.... 28
9. Repositório do Código.... 29
9.1 URL do repositório....29
9.2 Conteúdo do repositório.... 29
9.3 Histórico de commits....29
10. Considerações Finais....31
10.1 Resumo da entrega....31
10.2 Decisões arquiteturais documentadas (ADRs)....31
10.3 Possíveis evoluções para Fase 2....31
10.4 Aprendizados-chave.... 32

1. Introdução

1.1 Descrição do problema

Um grupo de restaurantes da região decidiu unir esforços para construir um sistema de gestão compartilhado, ao invés de cada estabelecimento contratar e manter um sistema próprio. A motivação é dupla: reduzir o custo individual com tecnologia e fazer com que a escolha do cliente passe a ser baseada na qualidade da comida — não na qualidade do sistema de cada restaurante.

O objetivo final é entregar um sistema robusto que permita aos restaurantes administrar suas operações com eficiência, enquanto os clientes consultam informações, deixam avaliações e fazem pedidos online. Em razão da limitação de recursos financeiros e do prazo apertado, ficou definido que o sistema seria entregue em fases — permitindo evolução gradual, validação contínua e ajustes antes de cada nova etapa.

Esta primeira fase trata exclusivamente do gerenciamento de usuários: o módulo que serve de fundação para todas as funcionalidades futuras (cardápio, pedidos, avaliações, pagamentos). Está fora do escopo desta fase qualquer integração externa, gateway de pagamento ou frontend.

1.2 Objetivo do projeto

Desenvolver um backend completo e robusto em Spring Boot, aplicando os princípios estudados na Fase 1 do curso, que ofereça as seguintes capacidades:

- Cadastro, atualização e exclusão de usuários;
- Endpoint dedicado para troca de senha, separado do endpoint de atualização de dados;
- Endpoint distinto para atualização das demais informações do usuário;
- Registro automático da data da última alteração em qualquer modificação;
- Busca de usuários pelo nome (case-insensitive, partial match);
- Garantia de unicidade do e-mail e do login no cadastro;
- Serviço de validação de login (login + senha) que confirme a autenticidade do usuário;
- Aplicação dockerizada com Docker Compose, orquestrando aplicação e banco relacional;
- Documentação Swagger/OpenAPI com exemplos de requisição e resposta para cada endpoint;
- Tratamento padronizado de erros conforme RFC 7807 (ProblemDetail);
- Versionamento da API sob o prefixo /api/v1.

Como diferencial e desafio extra previsto pelo enunciado, este projeto também implementa autenticação stateless via Spring Security 6 + JWT (HS256, JJWT 0.12) e cobertura abrangente de testes automatizados — unitários (JUnit 5 + Mockito), de fatia web (@WebMvcTest), de arquitetura (ArchUnit) e de integração ponta-a-ponta (Testcontainers contra PostgreSQL real).

1.3 Metodologia de desenvolvimento

O projeto foi conduzido com a abordagem Spec-Driven Development (SDD): antes de qualquer linha de código, foram redigidas as especificações de produto (visão, requisitos, critérios de aceite), as especificações técnicas (arquitetura, stack, convenções, tratamento de erros), os ADRs (Architectural Decision Records) e os specs detalhados de cada módulo de implementação (M01 a M11), com escopo, contratos de interfaces, regras e Definition of Done explícita.

A entrega foi feita em doze módulos sequenciais, cada um com commit próprio, suíte de testes verde e passagem completa do ArchUnit antes de avançar. Esta abordagem minimizou retrabalho, manteve a coerência arquitetural ao longo de toda a evolução e permitiu rastreabilidade total entre código e requisitos. Toda a documentação de especificação está versionada no repositório, dentro do diretório specs/.

2. Arquitetura do Sistema

2.1 Estilo arquitetural

O sistema adota a Arquitetura Hexagonal (Ports & Adapters) em sua forma pragmática: módulo Maven único, separação estrita por pacotes e regras de dependência validadas em tempo de build pelo ArchUnit. Esta decisão está formalizada no ADR-001 do projeto.

A motivação para esta escolha foi o cenário concreto do desafio: um desenvolvedor solo, prazo curto e a certeza de que as próximas fases adicionarão novos casos de uso (pedidos, cardápio, avaliações) sobre o mesmo backend. Uma arquitetura em camadas clássicas (controller → service → repository) seria mais simples no curto prazo, mas acoplaria a lógica de negócio ao Spring e ao JPA, dificultando a extensão futura. Um multi-módulo Maven seria mais puro, mas com sobrecarga de configuração desproporcional ao prazo. A hexagonal pragmática equilibra rigor arquitetural com produtividade.

2.2 Camadas e responsabilidades

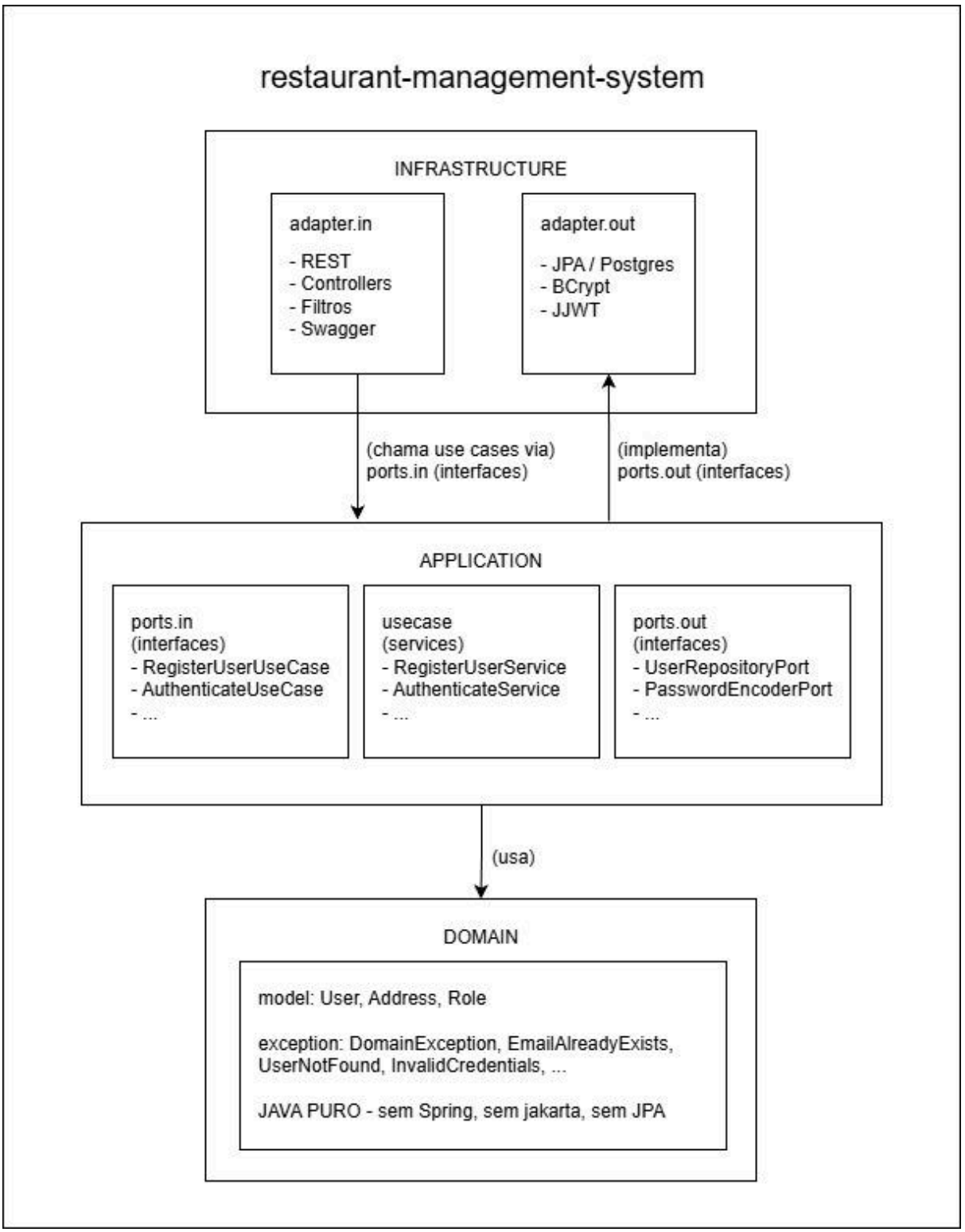
A aplicação é organizada em quatro camadas, com regras de dependência fortemente assimétricas: as setas só apontam para dentro (de infrastructure para application, e de application para domain). A camada shared pode ser usada por qualquer camada, mas não pode depender de nenhuma.

Camada	Responsabilidade	Pode depender de
domain	Entidades, value objects, exceções de negócio. Pure Java, zero framework.	Apenas JDK
application	Interfaces de casos de uso (ports.in), interfaces de repositórios e serviços externos (ports.out), implementações dos casos de uso. Sem framework.	domain + JDK
infrastructure	Adaptadores concretos: controllers REST, JPA, Spring Security, JWT, OpenAPI. É onde o Spring vive.	application, domain, shared
shared	Tipos transversais e o GlobalExceptionHandler para exceções de framework.	Nenhuma camada do projeto

Estas regras não são apenas convenções: oito testes ArchUnit em ArchitectureTest.java validam-nas a cada execução de mvn verify. Qualquer importação acidental de jakarta..*, org.springframework..* ou org.hibernate..* dentro de domain quebra o build imediatamente. O mesmo vale para controllers que tentem chamar adaptadores de saída diretamente (em vez de passar por um port).

2.3 Diagrama da arquitetura

O diagrama abaixo representa a estrutura hexagonal do sistema, com o domain no centro, application como camada de orquestração de casos de uso, e infrastructure contendo todos os adaptadores que conectam o sistema ao mundo externo (HTTP, banco de dados, biblioteca JWT, BCrypt).



2.4 Fluxo de uma requisição

Para tornar a teoria concreta, segue o caminho percorrido por uma requisição típica de cadastro de usuário (POST /api/v1/users):

- **Passo 1:** Cliente envia HTTP POST com JSON contendo os dados do novo usuário;

- **Passo 2:** JwtAuthenticationFilter (em infrastructure.adapter.in.web.security) inspeciona o header Authorization. Como /users de POST é público, o filtro deixa passar sem autenticar;
- **Passo 3:** UserController recebe a requisição, valida o RegisterUserRequest via Jakarta Bean Validation;
- **Passo 4:** UserWebMapper converte o DTO em RegisterUserCommand (objeto da camada application);
- **Passo 5:** Controller chama RegisterUserUseCase.register(command). A interface está em application.port.in — o controller não sabe qual é a implementação concreta;
- **Passo 6:** RegisterUserService (a implementação) consulta UserRepositoryPort para verificar unicidade de e-mail e login, chama PasswordEncoderPort.encode() para gerar o hash BCrypt, cria a entidade User via User.create(...) e persiste via userRepository.save();
- **Passo 7:** JpaUserRepositoryAdapter (em infrastructure.adapter.out.persistence) traduz o domínio para UserJpaEntity através do UserPersistenceMapper, persiste via Spring Data, e devolve a entidade traduzida de volta para o domínio;
- **Passo 8:** UserController converte o User retornado em UserResponse (sem campos de senha) e responde 201 Created com header Location apontando para o recurso criado.

Observe que em nenhum ponto deste fluxo o controller importa o adaptador JPA, e o caso de uso (RegisterUserService) não conhece o Spring nem o Jakarta. A inversão de dependência é total: o domain é o núcleo, application orquestra, infrastructure adapta.

2.5 Wiring de beans: domain agnóstico ao Spring

Uma decisão arquitetural importante decorre da Regra 2 do ArchUnit ("application não pode depender de Spring"): os serviços em application.usecase NÃO podem carregar a anotação @Service. Para resolver isso sem violar a regra, os casos de uso são POJOs registrados como beans através de métodos @Bean explícitos em UseCaseBeanConfiguration:

```
@Configuration
public class UseCaseBeanConfiguration {

    @Bean
    RegisterUserUseCase registerUserUseCase(
            UserRepositoryPort userRepository,
            PasswordEncoderPort passwordEncoder) {
        return new RegisterUserService(userRepository, passwordEncoder);
    }

    @Bean
    AuthenticateUserUseCase authenticateUserUseCase(
            AuthenticationStrategyPort authenticationStrategy,
            JwtTokenProviderPort jwtTokenProvider) {
        return new AuthenticateUserService(authenticationStrategy, jwtTokenProvider);
    }

    // ... demais use cases
}
```

O tipo declarado de retorno do bean é a interface (ex.: RegisterUserUseCase), não a implementação. Isso força os controllers a dependerem do port, mantém a camada application livre de Spring e ainda assim aproveita toda a injeção de dependência do framework. O padrão escala linearmente: adicionar um novo caso de uso é uma única declaração @Bean.

2.6 Estratégia plugável de autenticação

A autenticação foi modelada com Strategy Pattern no nível de port (ADR-002). A interface AuthenticationStrategyPort define o contrato authenticate(login, senha) → AuthenticationResult. Nesta fase existe um único adaptador, DatabaseAuthenticationAdapter, que valida credenciais contra o banco. Estratégias futuras (OAuth2, LDAP, autenticação externa) entram como novos adaptadores, sem qualquer alteração no caso de uso AuthenticateUserService.

A emissão do JWT é uma capacidade separada, modelada por outro port: JwtTokenProviderPort, com método generateToken(userId, role). O adaptador atual JjwtTokenProviderAdapter usa a biblioteca JJWT 0.12 e algoritmo HS256. Esta separação respeita o Single Responsibility Principle: o adaptador de credenciais não sabe nada sobre tokens, e o adaptador de tokens não sabe nada sobre senhas.

2.7 Tratamento de erros: ProblemDetail (RFC 7807)

Toda resposta de erro da API segue o padrão ProblemDetail (RFC 7807), com Content-Type application/problem+json. A estratégia foi formalizada em specs/technical/04-error-handling.md.

Existem dois @RestControllerAdvice no projeto, separados pela camada de origem da exceção tratada — uma necessidade derivada da Regra 4 do ArchUnit (shared não pode depender de domain):

Advice	Camada	Trata exceções de	Ordem
DomainExceptionHandler	infrastructure.adapter.in.web	domain.exception (UserNotFound, EmailAlreadyExists, LoginAlreadyExists, InvalidCredentials)	@Order(1)
GlobalExceptionHandler	shared.exception	Framework (MethodArgumentNotValidException, ConstraintViolationException, MissingServletRequestParameterException, Exception catch-all)	@Order(LOWEST_PRECEDENCE)

Aviso histórico do projeto: este ordenamento via @Order foi adicionado em M11 após a descoberta de um bug latente. Sem @Order explícito, o Spring resolve a ordem dos advices pela ordem de varredura do classpath — que é diferente entre testes em processo e o JAR empacotado. No JAR de produção, o catch-all @ExceptionHandler(Exception.class) do GlobalExceptionHandler estava

casando ANTES das exceções de domínio, transformando todo 4xx esperado (404, 409, 401) em 500. Os 100 testes automatizados passavam (eles rodam em outra ordem de classpath), mas a primeira execução real do Newman contra o JAR em Docker Compose revelou o problema. A correção foi adicionar @Order(1) ao DomainExceptionHandler e @Order(Ordered.LOWEST_PRECEDENCE) ao GlobalExceptionHandler. Lição registrada em `specs/technical/03-conventions.md`: todo @RestControllerAdvice deve declarar @Order explícito.

2.8 Estrutura de pacotes

```
com.fiap.rms
├── domain
│   ├── model              ← User, Address, Role
│   └── exception          ← DomainException + 6 exceções específicas
├── application
│   ├── port
│   │   ├── in             ← 7 interfaces de UseCase
│   │   └── out            ← UserRepository, PasswordEncoder, JwtTokenProvider,
│   │                         AuthenticationStrategy
│   └── usecase            ← 7 implementações + RegisterUserCommand,
│                             UpdateUserCommand, AuthenticationResult, TokenPayload
├── infrastructure
│   ├── adapter
│   │   ├── in
│   │   │   └── web        ← UserController, AuthController, DTOs,
│   │   │       └── security  ← JwtAuthenticationFilter, EntryPoint, AccessDenied
│   │   └── out
│   │       ├── persistence ← UserJpaEntity, repositório, adapter, mapper
│   │       └── security    ← BCryptAdapter, DatabaseAuthAdapter, JjwtAdapter
│   └── config             ← SecurityConfig, OpenApiConfig, JwtProperties,
│                             UseCaseBeanConfiguration
└── shared
    └── exception          ← GlobalExceptionHandler, NotFoundException
```

3. Modelagem de Dados

3.1 Modelo de entidade

Esta primeira fase exige apenas a modelagem da entidade User. A escolha de não dividir Address em uma tabela separada foi deliberada: enquanto cada usuário tiver exatamente um endereço, mantê-lo como Value Object inline na tabela users é mais simples e performático. Caso uma fase futura precise de múltiplos endereços por usuário (entrega, cobrança, etc.), a evolução natural é extrair Address para uma tabela própria com FK — registrado como nota técnica em NOTES.md.

3.2 Diagrama Entidade-Relacionamento

Como nesta fase existe apenas uma entidade persistente, o ER é trivialmente simples. Apresento-o em formato tabular detalhado para destacar tipos, restrições e índices definidos na migração Flyway:

Coluna	Tipo	Restrições	Descrição
id	UUID	PRIMARY KEY	Identificador único, gerado no domínio
name	VARCHAR(150)	NOT NULL	Nome completo do usuário
email	VARCHAR(255)	NOT NULL, UNIQUE (idx_users_email)	E-mail único no sistema
login	VARCHAR(60)	NOT NULL, UNIQUE (idx_users_login)	Login único para autenticação
password_hash	VARCHAR(255)	NOT NULL	Hash BCrypt da senha (cost 12)
role	VARCHAR(30)	NOT NULL	Papel: CUSTOMER, RESTAURANT_OWNER, ADMIN
street	VARCHAR(150)	NOT NULL	Logradouro do endereço
number	VARCHAR(20)	NOT NULL	Número do imóvel
city	VARCHAR(100)	NOT NULL	Cidade
zip_code	VARCHAR(20)	NOT NULL	CEP
created_at	TIMESTAMP WITH TIME ZONE	NOT NULL	Data/hora de criação (UTC)
updated_at	TIMESTAMP WITH TIME ZONE	NOT NULL	Data da última alteração (UTC) — atualizado a cada modificação

3.3 Migrações Flyway

O esquema é gerenciado por Flyway, com os scripts em src/main/resources/db/migration/. Há duas migrações:

- **`V1__create_users_table.sql` –** Cria a tabela users com todas as colunas, define a PK, o índice único em email e o índice em name.
- **`V2__add_unique_index_users_login.sql` –** Adiciona o índice único sobre login. Foi separada na V2 para refletir a evolução real do projeto: o requisito de unicidade de login (FR08) foi formalizado depois da migração inicial.

Em todos os profiles (dev, hom, prod) o Hibernate está configurado com ddl-auto: validate — o Flyway é a única fonte de verdade do esquema, e o Hibernate apenas valida que o esquema do banco corresponde às entidades JPA. Isso garante reprodutibilidade entre ambientes e zero risco de migrações implícitas em produção.

3.4 Decisões importantes

Timestamps com timezone: TIMESTAMP WITH TIME ZONE foi escolhido em vez de TIMESTAMP simples. O Java Instant é sempre UTC, e o tipo with time zone preserva essa informação no banco. TIMESTAMP simples descarta silenciosamente o offset, o que pode corromper round-trips quando o timezone do servidor difere de UTC.

Índice em name: originalmente o spec previa um índice funcional sobre LOWER(name) para acelerar a busca case-insensitive. O H2 2.x — usado no profile dev — não suporta índices em expressões mesmo no modo PostgreSQL. A solução foi um índice plano sobre name, com a busca case-insensitive resolvida pela query do Spring Data (LOWER(name) LIKE LOWER(?)).

UUID gerado no domínio: User.create() invoca UUID.randomUUID() antes de persistir. A entidade JPA (UserJpaEntity) recebe o UUID já populado. Isso permite usar o ID antes mesmo do save, simplifica testes e evita dependência do banco para identificação.

4. Descrição dos Endpoints da API

A API expõe 7 endpoints sob o prefixo /api/v1, conforme a convenção de versionamento exigida pelo desafio (NFR03). A tabela abaixo é o catálogo completo:

4.1 Tabela de endpoints

Método	Endpoint	Descrição	Auth
POST	`/api/v1/users`	Cadastrar novo usuário	Pública
GET	`/api/v1/users/{id}`	Buscar usuário por id	JWT
GET	`/api/v1/users?name={termo}`	Pesquisar usuários por nome (case-insensitive)	JWT
PUT	`/api/v1/users/{id}`	Atualizar dados (sem senha)	JWT
PATCH	`/api/v1/users/{id}/password`	Alterar senha (endpoint dedicado)	JWT
DELETE	`/api/v1/users/{id}`	Remover usuário	JWT
POST	`/api/v1/auth/login`	Validar credenciais e emitir JWT	Pública

Endpoints marcados como "Pública" não exigem token (registro inicial e login). Todos os demais exigem header Authorization: Bearer <jwt> válido — caso contrário, o filtro JwtAuthenticationFilter mantém a requisição não autenticada e o RestAuthenticationEntryPoint responde 401 com ProblemDetail.

4.2 Exemplos de requisição e resposta

4.2.1 POST /api/v1/users — cadastro de usuário

Request:

```
POST /api/v1/users
Content-Type: application/json

{
  "name": "Maria da Silva",
  "email": "maria@exemplo.com.br",
  "login": "mariasilva",
  "password": "Senha@123",
  "role": "CUSTOMER",
  "address": {
    "street": "Rua das Flores",
    "number": "42",
    "city": "São Paulo",
    "zipCode": "01310-100"
  }
}
```

Resposta 201 Created (sucesso):

```
HTTP/1.1 201 Created
Location: /api/v1/users/3fa85f64-5717-4562-b3fc-2c963f66afa6
Content-Type: application/json

{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "name": "Maria da Silva",
  "email": "maria@exemplo.com.br",
  "login": "mariasilva",
  "role": "CUSTOMER",
  "address": {
    "street": "Rua das Flores",
    "number": "42",
    "city": "São Paulo",
    "zipCode": "01310-100"
  },
  "createdAt": "2026-04-30T10:00:00Z",
  "updatedAt": "2026-04-30T10:00:00Z"
}
```

Resposta 409 Conflict (e-mail já cadastrado):

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://api.techchallenge.com/errors/email-conflict",
  "title": "E-mail já cadastrado",
  "status": 409,
  "detail": "Já existe um usuário com o e-mail informado.",
  "instance": "/api/v1/users",
  "timestamp": "2026-04-30T12:00:00Z"
}
```

Resposta 400 Bad Request (campo obrigatório ausente):

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "https://api.techchallenge.com/errors/validation",
  "title": "Dados inválidos",
  "status": 400,
  "detail": "Um ou mais campos estão inválidos.",
  "instance": "/api/v1/users",
  "timestamp": "2026-04-30T12:00:00Z",
  "errors": [
    { "field": "name", "message": "não deve estar em branco" }
  ]
}
```

4.2.2 POST /api/v1/auth/login — autenticação

Request:

```
POST /api/v1/auth/login
Content-Type: application/json

{
  "login": "mariasilva",
  "password": "Senha@123"
}
```

Resposta 200 OK (sucesso):

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "authenticated": true,
  "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "role": "CUSTOMER",
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIzZmE4...",
  "expiresIn": 3600
}
```

Resposta 401 Unauthorized:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/problem+json

{
  "type": "https://api.techchallenge.com/errors/unauthorized",
  "title": "Não autenticado",
  "status": 401,
  "detail": "Credenciais inválidas.",
  "instance": "/api/v1/auth/login",
  "timestamp": "2026-04-30T12:00:00Z"
}
```

Importante: tanto "login não existe" quanto "senha incorreta" retornam o MESMO corpo de resposta — byte por byte, exceto pelo timestamp. Isso evita o vazamento de informação sobre quais logins existem no sistema (user enumeration attack). O comportamento é validado por teste automatizado em AuthControllerWebMvcTest.

4.2.3 PATCH /api/v1/users/{id}/password — alteração de senha

Endpoint dedicado, separado da atualização de dados — exigência explícita do desafio (FR03). A senha não pode ser trocada via PUT /users/{id}, e o DTO UpdateUserRequest sequer possui um campo password (Jackson ignora o campo extra silenciosamente, comportamento validado por teste).

Request:

```
PATCH /api/v1/users/3fa85f64-5717-4562-b3fc-2c963f66afa6/password
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{
  "newPassword": "NovaSenha@1"
}
```

Resposta 204 No Content (sucesso, sem corpo):

```
HTTP/1.1 204 No Content
```

A senha precisa atender a três regras: mínimo 8 caracteres, no máximo 72 (limite do BCrypt), e conter pelo menos uma letra maiúscula e um dígito. O regex de validação é (?=.*[A-Z])(?=.*\d). Senhas fracas retornam 400 com ProblemDetail.

4.2.4 GET /api/v1/users?name={termo} — busca por nome

Request:

```
GET /api/v1/users?name=maria
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

Resposta 200 OK com resultados:

```
HTTP/1.1 200 OK
Content-Type: application/json

[
  {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "name": "Maria da Silva",
    "email": "maria@exemplo.com.br",
    ...
  },
  {
    "id": "7b3a85f64-...",
    "name": "MARIA SOUZA",
    ...
  }
]
```

A busca é case-insensitive e usa partial match (LIKE %termo%). Quando nenhum usuário atende ao critério, a resposta é 200 OK com array vazio []. Esta distinção é deliberada (e validada por teste): array vazio significa "nenhum usuário encontrado"; 404 significaria "endpoint não existe". A ausência de paginação é decisão consciente — fora do escopo da Fase 1.

4.2.5 Demais endpoints

Para evitar repetição, os endpoints restantes seguem padrões semelhantes aos exemplos acima:

- **`GET /api/v1/users/{id}`** — Retorna o UserResponse completo (200) ou ProblemDetail com type=/errors/user-not-found (404).
- **`PUT /api/v1/users/{id}`** — Substitui name, email, login e address. Retorna 200 com o UserResponse atualizado, 404 se não existe, 409 se email/login colidem com outro usuário, 400 em validação. updatedAt é refrescado.
- **`DELETE /api/v1/users/{id}`** — Remoção definitiva (hard delete). 204 No Content em sucesso, 404 se inexistente.

4.3 Códigos HTTP utilizados

Status	Significado no projeto
201 Created	Criação bem-sucedida (POST /users), com header Location
200 OK	Leitura ou atualização bem-sucedida (GET, PUT, POST /auth/login)
204 No Content	Sucesso sem corpo de resposta (DELETE, PATCH /password)

Status	Significado no projeto
400 Bad Request	Falha de validação (Jakarta Bean Validation)
401 Unauthorized	Falha de autenticação (token ausente, inválido, expirado, ou credenciais erradas)
404 Not Found	Recurso solicitado não existe
409 Conflict	Violação de unicidade (e-mail ou login duplicados)
500 Internal Server Error	Erro inesperado — log completo no servidor, mensagem genérica para o cliente, sem stack trace

5. Configuração do Projeto

5.1 Stack e versões

Componente	Versão
Java	21 (LTS)
Maven	3.9+
Spring Boot	3.3.5
Spring Security	6.x (stateless, filtro JWT)
JJWT (HS256)	0.12.6
Spring Data JPA + Hibernate	(gerenciado pelo Boot)
Flyway	10.x + flyway-database-postgresql
PostgreSQL	16-alpine (Docker)
H2 (apenas no profile dev)	(gerenciado pelo Boot)
springdoc-openapi	2.6.0
Testcontainers	1.20.4
ArchUnit	1.3.0
Lombok	Provided (apenas em infrastructure)

5.2 Profiles Spring

Profile	Banco	Swagger	Logs	Uso
dev	H2 in-memory (modo PostgreSQL)	Habilitado	DEBUG	Desenvolvimento local rápido, sem Docker
hom	PostgreSQL (Docker)	Habilitado	INFO	Homologação (default no Docker Compose)
prod	PostgreSQL	Desabilitado	INFO	Produção — JWT_SECRET default explicitamente rejeitado no startup

5.3 Configuração do Docker Compose

O arquivo docker-compose.yml na raiz do repositório orquestra dois serviços: o banco PostgreSQL e a aplicação Spring Boot. A imagem da aplicação é construída a partir do Dockerfile multi-stage do projeto.

```
services:

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: techchallenge
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5

  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      SPRING_PROFILES_ACTIVE: hom
      DB_URL: jdbc:postgresql://db:5432/techchallenge
      DB_USERNAME: postgres
      DB_PASSWORD: postgres
      JWT_SECRET: "${JWT_SECRET:?JWT_SECRET is required - generate with: openssl rand
-base64 48}"
      JWT_EXPIRATION_SECONDS: 3600
    depends_on:
      db:
        condition: service_healthy

volumes:
  pgdata:
```

Pontos relevantes desta configuração:

- **Healthcheck no banco:** O serviço app depende do db com condition: service_healthy. A aplicação só inicia depois que pg_isready confirma que o PostgreSQL está aceitando conexões. Isso elimina race conditions na primeira execução.
- **Fail-fast no JWT_SECRET:** A sintaxe ${JWT_SECRET:?...} faz o Compose abortar antes mesmo de subir os containers caso a variável não esteja definida, com mensagem orientando como gerar uma (openssl rand -base64 48). Sem isso, a aplicação só falharia depois de subir, gastando tempo de feedback.
- **Volume persistente:** O volume pgdata mantém os dados do PostgreSQL entre execuções. Para começar do zero (útil ao rodar a coleção Postman repetidamente), basta executar docker compose down -v.

5.4 Dockerfile multi-stage

O Dockerfile usa duas etapas: a primeira compila o JAR com Maven dentro de uma imagem Maven oficial; a segunda roda apenas o JAR sobre um JRE Alpine — significativamente menor que a imagem de build.

```
# Stage 1: build
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /build

# Copia o POM primeiro para aproveitar cache de dependências
COPY pom.xml .
RUN mvn -B dependency:go-offline

COPY src ./src
RUN mvn -B -DskipTests package

# Stage 2: runtime
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

COPY --from=build /build/target/*.jar /app/app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

5.5 Variáveis de ambiente

Variável	Obrigatória	Default	Descrição
SPRING_PROFILES_ACTIVE	Sim	hom (no Compose)	dev / hom / prod
DB_URL	hom / prod	—	JDBC URL (ex: jdbc:postgresql://db:5432/techchallenge)
DB_USERNAME	hom / prod	—	Usuário do banco
DB_PASSWORD	hom / prod	—	Senha do banco
JWT_SECRET	Sim	— (Compose falha)	Segredo HS256, mínimo 32 bytes UTF-8
JWT_EXPIRATION_SECONDS	Não	3600	Tempo de vida do token, em segundos

5.6 Instruções para execução

5.6.1 Execução local com profile dev (H2)

Útil para desenvolvimento rápido — não requer Docker. O profile dev usa H2 in-memory em modo de compatibilidade PostgreSQL.

```
# 1. Definir o segredo JWT (obrigatório mesmo em dev)
export JWT_SECRET=$(openssl rand -base64 48)

# 2. Subir a aplicação
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev
```

URLs disponíveis:

- **API:** `http://localhost:8080`
- **Swagger UI:** `http://localhost:8080/swagger-ui.html`
- **H2 Console:** `http://localhost:8080/h2-console`
- **Health:** `http://localhost:8080/actuator/health`

5.6.2 Execução com Docker Compose (PostgreSQL real)

```
# 1. Copiar .env.example
cp .env.example .env

# 2. Editar .env: substituir JWT_SECRET por valor gerado com:
#    openssl rand -base64 48

# 3. Subir tudo (build + app + db)
docker compose up --build
```

Aguarde até o log mostrar Started RmsApplication. A API estará em http://localhost:8080.

5.6.3 Resetar dados entre execuções

A coleção Postman insere usuários com identificadores fixos (admin, customer, owner). Ao rerodar a coleção sem limpar o banco, esses inserts retornam 409 Conflict, derrubando os passos seguintes. Para começar do zero:

```
docker compose down -v   # -v remove o volume pgdata
docker compose up --build
```

6. Qualidade do Código e Testes

6.1 Boas práticas adotadas

- **SOLID em primeira linha:** Cada caso de uso é uma classe pequena com uma única responsabilidade (Single Responsibility). A inversão de dependência é natural — controllers dependem de interfaces (ports.in), nunca de implementações. O princípio Aberto/Fechado se manifesta na estratégia plugável de autenticação: novos métodos de auth entram como adaptadores, sem tocar no caso de uso.
- **DRY com moderação:** Mappers (UserWebMapper, UserPersistenceMapper) centralizam a tradução entre representações; o handler de exceções centraliza a serialização do ProblemDetail. Por outro lado, os ITs Testcontainers foram deliberadamente NÃO refatorados em uma classe base comum — manter cada IT autocontido facilita o diagnóstico de falhas, ainda que duplique trechos de configuração.
- **Convenções do Spring Boot:** Uso idiomático de @SpringBootApplication, @RestController, @RestControllerAdvice, @ConfigurationProperties (binding de JwtProperties como record). Validação por Jakarta Bean Validation nos DTOs (@NotBlank, @Email, @Size, @Pattern, @Valid em campos compostos). Atributo de profile via @Profile no ErrorProbeController.
- **Imutabilidade no domínio:** User só muda estado por métodos comportamentais (updateProfile, changePassword), nunca por setters. Address é record. Role é enum. equals/hashCode baseados apenas no id. toString omite o passwordHash deliberadamente.
- **Nomenclatura disciplinada:** Casos de uso seguem <Acao><Entidade>UseCase / <Acao><Entidade>Service. Comandos: <Acao><Entidade>Command. Ports: <Capacidade>Port. Adaptadores: <Tecnologia><Capacidade>Adapter. JPA entities: <Entidade>JpaEntity. DTOs: <Acao><Entidade>Request / <Entidade>Response.
- **Idioma das mensagens:** Código, identificadores, comentários e mensagens de commit em inglês — alinhado às convenções de open-source. Mensagens user-facing (ProblemDetail title/detail, validações) em português brasileiro.

6.2 Estratégia de testes

O projeto totaliza mais de 100 testes verdes, distribuídos em 4 categorias com objetivos distintos. A meta de cobertura é ≥ 80% em domain e application, e a estratégia segue a pirâmide de testes: muitos testes unitários rápidos, alguns testes de integração de slice e poucos testes ponta-a-ponta.

Categoria	Ferramentas	Quantos	O que validam
Testes unitários	JUnit 5, Mockito, AssertJ	~50	Lógica de domínio (User, Address) e de caso de uso (Register/Update/Delete/etc). Sem contexto Spring.
Testes de slice web	@WebMvcTest + Spring Security Test	~30	Camada HTTP isolada: validação de DTOs, status codes, mapeamento exception →

Categoria	Ferramentas	Quantos	O que validam
			ProblemDetail. Casos de uso são mockados.
Testes de arquitetura	ArchUnit	8 regras	Garantem que regras de dependência, nomenclatura de UseCase/Port/Adapter e isolamento do domain não são violados.
Testes de integração	Testcontainers + PostgreSQL real	~10	Fluxos completos HTTP→Controller→UseCase→Adapter JPA→Postgres. Inclui ITs do CRUD, busca case-insensitive, autenticação ponta-a-ponta e fluxo segurado completo.

6.3 ArchUnit: 8 regras como guardião da arquitetura

A classe ArchitectureTest implementa oito regras executadas em mvn verify. Cada regra carrega uma cláusula .because(...) explicativa para que falhas tragam contexto:

- Regra 1: domain não pode depender de Spring, Jakarta, Hibernate, Lombok, ou de application/infrastructure;
- Regra 2: application só pode depender de domain e do JDK;
- Regra 3: domain e application não podem depender de infrastructure (setas só apontam para dentro);
- Regra 4: shared não pode depender de nenhuma camada do projeto;
- Regra 5: Controllers (web) não podem depender diretamente de adaptadores de saída — devem passar por ports;
- Regra 6: Tipos terminados em UseCase devem ser interfaces e residir em application.port.in;
- Regra 7: Tipos terminados em Port devem ser interfaces e residir em application.port.in ou application.port.out;
- Regra 8: Tipos terminados em Adapter devem residir em infrastructure.adapter..*

Estas regras evoluíram naturalmente com o projeto. A Regra 2 forçou a decisão de não usar @Service nos casos de uso (e adoção do UseCaseBeanConfiguration). A Regra 4 forçou a separação dos dois @RestControllerAdvice. A Regra 5 elimina a tentação de chamar JpaUserRepositoryAdapter diretamente do controller. Toda restrição arquitetural é uma escolha que evita uma classe inteira de bugs futuros.

6.4 Padrões específicos descobertos durante o desenvolvimento

O arquivo NOTES.md do repositório registra decisões e descobertas feitas durante a implementação de cada módulo. Algumas das mais relevantes:

@WebMvcTest com @Component colaboradores: mappers stateless (UserWebMapper) devem ser carregados via @Import(UserWebMapper.class), NUNCA via @MockBean. Mockar um mapper neutraliza o teste — a fatia precisa exercitar o mapeamento real. Apenas use cases (que carregam comportamento) são mockados.

ITs com Testcontainers + driver override: ao sobrescrever a datasource via @DynamicPropertySource, é preciso também sobrescrever spring.datasource.driver-class-name=org.postgresql.Driver. Sem isso, o profile dev vence e a aplicação tenta conectar com driver H2 em uma URL Postgres.

Testcontainers 1.20.4 + api.version=1.44: o Spring Boot BOM gerência Testcontainers em 1.19.x, cuja versão de docker-java embarcada usa API 1.41 — rejeitada pelo Docker Desktop 4.28+. Foi necessário override explícito da versão e configurar -Dapi.version=1.44 no argLine do failsafe-plugin.

Bug latente do @Order: documentado na seção 2.7 deste relatório. Surfou apenas no Newman contra o JAR empacotado. Lição institucional registrada em specs/technical/03-conventions.md: sempre @Order explícito em @RestControllerAdvice.

7. Documentação Swagger / OpenAPI

7.1 Configuração

A documentação OpenAPI 3.1 é gerada automaticamente pela biblioteca springdoc-openapi-starter-webmvc-ui 2.6.0. A configuração customizada vive em OpenApiConfig (em infrastructure.config) e define:

- Metadados da API (título, versão 1.0.0, descrição, contato);
- Servidores: localhost:8080 (desenvolvimento) e placeholder de homologação;
- Esquema de segurança bearerAuth (HTTP/JWT) que habilita o botão Authorize do Swagger UI;
- Schemas reutilizáveis ProblemDetail e ValidationProblemDetail — registrados como componentes para serem referenciados via $ref nas respostas de erro.

7.2 Anotações em controllers e DTOs

Cada operação é documentada com @Operation (resumo + descrição), @ApiResponses cobrindo todos os status codes possíveis, e @ExampleObject com payloads realistas em português brasileiro (nomes brasileiros, CEPs no formato NNNNN-NNN). DTOs marcam cada campo com @Schema(description, example), o que melhora consideravelmente a navegação no Swagger UI.

Endpoints públicos (POST /api/v1/auth/login, POST /api/v1/users) carregam @SecurityRequirements({}) — vazio — para sobrescrever o requisito global de Bearer e indicar ao Swagger UI que esses endpoints não exigem token.

7.3 Acesso à interface

Swagger UI: http://localhost:8080/swagger-ui.html (habilitado em dev e hom)

OpenAPI JSON: http://localhost:8080/v3/api-docs

No profile prod ambos são desabilitados via configuração em application-prod.yml — a documentação não vaza para produção.

7.4 Fluxo de uso pelo Swagger UI

Para chamar um endpoint protegido diretamente da interface:

- 1. Expandir o endpoint POST /api/v1/auth/login e executá-lo com credenciais válidas;
- 2. Copiar o valor do campo token da resposta;
- 3. Clicar no botão Authorize (canto superior direito da tela);
- 4. Colar o token no campo bearerAuth e confirmar — o Swagger anexa Authorization: Bearer <token> em todas as requisições subsequentes;
- 5. Chamar qualquer endpoint protegido normalmente.

7.5 Validação automatizada

O teste OpenApiSmokeTest valida programaticamente o documento /v3/api-docs:

- Os 7 endpoints estão presentes (POST/GET/PUT/PATCH/DELETE em /users, GET em /users/{id}, GET em /users com query, POST em /auth/login);
- Os schemas ProblemDetail e ValidationProblemDetail existem em components.schemas;
- O securityScheme bearerAuth está declarado com scheme=bearer e bearerFormat=JWT;
- Nenhum schema (de request ou response) contém o campo password ou passwordHash;
- POST /api/v1/auth/login tem array security vazio (override de @SecurityRequirements({})).

8. Coleção Postman para Testes

8.1 Estrutura da coleção

A coleção tech-challenge.postman_collection.json (no diretório postman/ do repositório) contém 25 requisições com 66 assertions, organizadas em 7 pastas executadas em ordem. A coleção define autenticação Bearer no nível raiz, com valor {{token}} — um placeholder preenchido automaticamente pelo script de teste do login.

Pasta	Conteúdo	Requests
0 — Auth setup	Cadastra o admin e captura o JWT em {{token}}. DEVE rodar primeiro.	4
1 — User registration	Cadastros válidos (customer + owner), além de cenários de erro: e-mail duplicado, login duplicado, campo ausente, role inválida.	6
2 — Update user data	PUT do admin (com verificação de updatedAt > createdAt), PUT de usuário inexistente (404), PUT enviando campo password (ignorado), login com senha antiga após o update (validação cruzada), PUT com login duplicado (409).	5
3 — Change password	PATCH válido, login com nova senha (smoke), PATCH em usuário inexistente, PATCH com senha fraca.	4
4 — Delete user	DELETE existente (204) e DELETE inexistente (404).	2
5 — Search by name	Busca com múltiplos resultados, busca sem resultados (200 + array vazio), busca case-insensitive ("ADMIN" encontra "Tech Admin Updated").	3
6 — Authorization edge cases	Endpoint protegido sem header Authorization (401), com Bearer malformado (401), e com token válido em estrutura mas assinatura errada (401).	3

8.2 Variáveis de ambiente

O arquivo tech-challenge.postman_environment.json define 6 variáveis. Apenas baseUrl é preenchida inicialmente; as demais são populadas durante a execução pelos scripts de teste:

- **`baseUrl:`** http://localhost:8080/api/v1 — endpoint base
- **`token:`** JWT capturado do login do admin (Folder 0)
- **`adminId:`** UUID do admin, capturado no cadastro (Folder 0)
- **`customerId:`** UUID do customer, capturado no Folder 1
- **`ownerId:`** UUID do restaurant_owner, capturado no Folder 1

- **`adminCreatedAt:`** Timestamp do cadastro do admin (usado no Folder 2 para confirmar que o updatedAt avança)

8.3 Importação

No Postman Desktop:

- Abrir Postman → Import;
- Selecionar os dois arquivos do diretório postman/: a coleção e o environment;
- No dropdown de environment (canto superior direito), selecionar Tech Challenge — User Management API (local);
- Abrir o Collection Runner, selecionar a coleção e executar com all folders in order — Folder 0 PRECISA rodar primeiro.

8.4 Execução com Newman (linha de comando)

Newman é o runner CLI do Postman. Útil para CI/CD e para validar a entrega ponta-a-ponta sem abrir a interface gráfica:

```
# Instalar Newman uma vez
npm install -g newman

# Subir a aplicação (em outro terminal)
docker compose up

# Rodar a coleção completa
newman run postman/tech-challenge.postman_collection.json \
  -e postman/tech-challenge.postman_environment.json
```

Foi exatamente esta execução do Newman contra o JAR de produção (e não contra os testes Spring rodando em IDE) que detectou o bug de ordenação dos @RestControllerAdvice descrito na seção 2.7. O cenário ressalta o valor de testes ponta-a-ponta na configuração real de delivery — uma camada que os testes em processo do projeto, mesmo com Testcontainers, não cobre completamente.

8.5 Cobertura dos critérios de aceite

Cada critério listado em specs/product/03-acceptance-criteria.md tem uma requisição correspondente verde na coleção. Os 35+ critérios cobrem cadastro válido e inválido, atualização (incluindo a impossibilidade de trocar senha pelo PUT), troca de senha (incluindo validação de nova senha), exclusão, busca por nome (com casos de zero/múltiplos resultados e case-insensitivity), validação de login (com paridade entre login inexistente e senha errada), e todos os edge cases de autenticação.

9. Repositório do Código

9.1 URL do repositório

O código-fonte completo, com README, especificações, ADRs, scripts Docker, coleção Postman e documentação Swagger gerada, está hospedado no GitHub: https://github.com/g-andradd/restaurant-management-system

O repositório é público e está estruturado para reprodutibilidade total — uma pessoa fazendo clone limpo segue os passos do README e tem a API rodando em menos de 5 minutos.

9.2 Conteúdo do repositório

Caminho	Conteúdo
src/main/java/	Código-fonte da aplicação organizado em 4 camadas (domain, application, infrastructure, shared)
src/main/resources/	application.yml, application-{dev,hom,prod}.yml, migrações Flyway em db/migration/
src/test/java/	Suíte de testes: unitários, ArchUnit, @WebMvcTest e ITs com Testcontainers
specs/	Especificações Spec-Driven Development: produto, técnico, ADRs, módulos M01–M12
postman/	Coleção e environment Postman (JSON)
report/	Este relatório técnico (PDF/DOCX)
pom.xml	POM Maven com BOM Spring Boot e dependências fixadas
Dockerfile	Multi-stage (Maven build → JRE runtime)
docker-compose.yml	Orquestração app + PostgreSQL com healthcheck e fail-fast no JWT_SECRET
.env.example	Modelo de variáveis de ambiente — copiar para .env e ajustar
README.md	11 seções cobrindo arquitetura, stack, execução local/Docker, testes, Postman e estrutura do projeto
NOTES.md	Diário técnico com decisões e descobertas por módulo (preserva o contexto histórico)

9.3 Histórico de commits

O histórico segue o padrão Conventional Commits, com um commit por módulo de implementação. A sequência reflete a ordem de construção do projeto:

- **`feat(M01):`** project skeleton

- **`feat(M02):`** architecture guardrails
- **`feat(M03):`** global error handling with ProblemDetail
- **`feat(M04):`** user domain model
- **`feat(M05):`** user persistence adapter
- **`feat(M06):`** user registration endpoint
- **`feat(M07):`** user read and search endpoints
- **`feat(M08):`** update, password change and delete
- **`feat(M09):`** authentication with JWT issuance
- **`feat(M09b):`** spring security jwt filter
- **`feat(M10):`** openapi/swagger documentation
- **`feat(M11):`** docker, postman collection and readme
- **`docs(M12):`** technical report

10. Considerações Finais

10.1 Resumo da entrega

O projeto entrega 100% dos requisitos funcionais (FR01 a FR08) e não funcionais (NFR01 a NFR08) listados em specs/product/02-requirements.md. Adicionalmente, foram implementados os dois itens marcados como desafio extra/opcional pelo enunciado:

- Spring Security 6 + JWT (HS256, JJWT 0.12.6), com filtro stateless, ProblemDetail customizado para 401/403 e proteção real de todos os endpoints (exceto cadastro inicial e login);
- Suíte robusta de testes unitários (JUnit 5 + Mockito), de fatia (@WebMvcTest), de arquitetura (ArchUnit) e de integração ponta-a-ponta (Testcontainers + Newman) — totalizando mais de 100 cenários verdes.

10.2 Decisões arquiteturais documentadas (ADRs)

Quatro decisões arquiteturais foram formalizadas como ADR no repositório, garantindo rastreabilidade do "porquê" por trás de cada escolha estrutural:

- **`ADR-001` – Pragmatic Hexagonal Architecture:** Adoção da arquitetura hexagonal em formato pragmático, módulo Maven único, com regras enforced via ArchUnit.
- **`ADR-002` – Pluggable Authentication:** Strategy Pattern no nível de port para autenticação, antecipando estratégias futuras (OAuth2, LDAP).
- **`ADR-003` – H2 in dev, PostgreSQL in hom/prod:** Mesma migração SQL ANSI funciona nos dois bancos; feedback rápido em dev, banco real em hom/prod.
- **`ADR-004` – Spring Security with JWT:** Antecipação do desafio extra de JWT para a Fase 1, justificada como signal de portfólio.

10.3 Possíveis evoluções para Fase 2

A arquitetura escolhida foi pensada para acomodar as próximas fases sem refatoração estrutural. Pontos de evolução natural identificados:

- **Multiplos endereços por usuário:** Hoje Address é Value Object inline. Para suportar endereços de entrega/cobrança separados, extrair para uma tabela addresses com FK.
- **Soft delete:** DELETE atual é hard delete. Quando entidade Order chegar com FK para users, será necessário trocar para soft delete (coluna deleted_at) para evitar violação de FK ou cascade indesejado.
- **Autorização por role:** A camada de autorização atualmente é coarse-grained (qualquer usuário autenticado acessa qualquer endpoint). O enum Role já tem ADMIN — basta adicionar regras hasRole(...) em SecurityConfig conforme as fases futuras exijam.

- **Separação de UserController:** Já controla 6 casos de uso. Em Fase 2, considerar split em UserCommandController + UserQueryController (CQRS leve).
- **Refresh tokens:** Token JWT atual é único, validade de 1h. Para sessões longas, adicionar refresh token e adaptar JwtTokenProviderPort.

10.4 Aprendizados-chave

Três descobertas se destacam pelo valor pedagógico:

Spec-Driven Development funciona em projeto solo. Investir nas especificações antes de codar evitou retrabalho, manteve coerência ao longo de 12 módulos e produziu um repositório que se documenta a si mesmo. O custo inicial de escrever specs detalhadas pagou-se já no terceiro módulo.

ArchUnit é o melhor revisor possível. Cada regra arquitetural escrita como teste impediu, ao longo do projeto, pelo menos uma tentativa acidental de violar a arquitetura — e cada falha trouxe contexto via .because(...). É um dos artefatos de mais alto retorno por linha escrita.

Testes ponta-a-ponta na configuração real de delivery são indispensáveis. Os 100+ testes automatizados estavam verdes, e mesmo assim o Newman contra o JAR empacotado (em Docker Compose, com PostgreSQL real) revelou um bug crítico de ordenação de @RestControllerAdvice. Lição: cobertura completa exige rodar a aplicação no formato em que ela será entregue, não apenas no formato em que ela é desenvolvida.

— Fim do relatório —